

chapter9

제9장 실험 추적과 모델 레지스트리

8장까지 만든 것은 데이터와 피쳐였다 — 수집(4장), 중복 제거(5장), 실시간 집계(6장), 배치 확정(7장), 그리고 훈련·서빙이 같은 값을 사용하게 하는 피쳐 계약(8장). 이 장에서 처음으로 **모델 훈련**이 등장한다. 그리고 훈련이 등장하는 순간 새 질문이 따라온다: “그 모델을 어떻게 만들었는가.” 어느 데이터로, 어느 피쳐로, 어느 파라미터로, 어느 코드로 — 이 넷에 답하지 못하면 그 모델은 재현할 수도, 감사할 수도, 안심하고 교체할 수도 없다. 이 장은 그 답을 사람의 기억이 아니라 **기록 시스템**에 두는 방법을 다룬다. MLflow로 실험을 기록하고(파라미터·메트릭·아티팩트), 같은 설정의 재실행이 같은 결과를 내는지 실측으로 검증하고, 배포 후보를 모델 레지스트리에 버전으로 등록해 승격(champion 별칭)을 게이트로 판정한다. 실습(★★★☆☆)의 훈련 입력은 8장 피쳐(지역구별 전일 민원 건수) 그대로다 — 7장의 재현성 원칙 “같은 입력 → 같은 산출물”이 이 장에서 “같은 실험 설정 → 같은 모델”로 확장된다.

학습 목표

실험 추적의 필요성과 핵심 기록 항목을 이해한다.
MLflow로 실험 파라미터와 지표를 기록한다.
모델 레지스트리와 승격 절차를 설명한다.
요구사항에서 승격 게이트 규칙(판정 지표·임계값·상대 조건·승인 주체)을 도출하고, 각 값의 근거와 그 규칙이 깨지는 조건을 적을 수 있다.
같은 기록 규율이 민간 실험 문화에서 어떤 지표로 쓰이는지 비교하고, 근거만 도메인이 바뀐다는 것을 설명할 수 있다.

9.1 실험 재현성의 의미

학습 목표

“재현할 수 없는 모델”이 기본값인 이유와 재현 실패의 4원인을 안다.
훈련을 입력의 함수로 보는 관점(7장의 확장)을 이해한다.

직관: 답할 수 없는 상태가 기본값이다

석 달 전에 배포한 민원 급증 예측 모델의 성능이 떨어졌다고 가정한다. 자연스러운 첫 조치는 “그때 그 모델을 다시 만들어 지금 데이터와 비교”하는 것이다. 그런데 다시 만들려면 알아야 한다 — 그때 어느 데이터를 사용했는가(7월 확정 집계인가, 정정 반영 전인가), 어느 피쳐 정의였는가

(complaint_model_v1인가 그 전인가), 학습 파라미터는 무엇이었는가, 코드는 어느 커밋이었는가. 아무것도 기록하지 않았다면 이 넷 중 하나도 확실하지 않고, “다시 만든 모델”은 그때 모델의 재현이 아니라 **새 모델**이다 — 비교 실험 자체가 성립하지 않는다.

주의할 점은 이것이 게으름의 결과가 아니라는 것이다. 실험은 본성상 빠르게 변한다 — 하루에 파라미터를 다섯 번 바꾸고, 데이터를 두 번 교체하는 것이 정상적인 실험 과정이다. 그 속도에서 “나중에 정리해야지”는 반드시 실패한다. 그래서 기록은 실험 **후**의 문서화가 아니라 실험 **중**의 자동 기록이어야 하고, 그것이 실험 추적 도구가 존재하는 이유다.

훈련도 함수다 — 7장 재현성의 확장

7장은 배치를 “입력의 결정적 함수”로 만들었다: 같은 입력 파일이면 같은 산출물, 해시까지 동일했다(7장 실측 — 두 실행의 sha256 일치). 훈련도 같은 관점으로 볼 수 있다: **모델 = f(코드, 데이터, 파라미터, 환경)**. 네 입력이 전부 같으면 같은 모델이 나와야 한다. 다만 훈련에는 배치에 없던 입력이 하나 숨어 있다 — 난수다. 초기화·셔플·샘플링이 난수를 쓰는 알고리즘이라면 seed까지 입력에 포함해 고정해야 함수가 된다(실습의 두 후보는 결정적 알고리즘이라 seed 없이도 결정적이다 — 9.2에서 실측으로 확인한다).

재현 실패는 네 입력 중 무엇이 달라졌는지로 분류된다.

표 9.1 재현 4요소와 기록 수단(실습의 구현)

요소	재현 실패의 전형	실습의 기록 수단
코드	훈련 스크립트가 그새 수정됨	버전 관리(git) — 실습 저장소 자체
데이터	원천이 정정·추가로 변함	데이터 지문 (sha256)을 파라미터로 기록
파라미터	“그때 뭘 줬더라”	run에 <code>log_params</code> 로 자동 기록
환경	라이브러리 버전 차이	모델과 함께 의존성 기록 (MLflow가 동봉), 증거 파일에 버전 명시

넷 중 실무에서 가장 자주 놓치는 것이 데이터다. 코드는 git이 지키고 파라미터는 도구가 지키지만, “어느 데이터였는가”는 도구가 자동으로 보장하지 않는다 — 훈련 데이터는 파일 경로가 같아도 내용이 바뀔 수 있기 때문이다(7장의 정정 재실행을 참조한다). 실습이 훈련 데이터의 해시를 파라미터로 함께 기록하는 이유이며(9.2), 이것은 5장(이벤트 지문)·7장(산출물 해시)에서 쓴 지문 기법을 훈련 데이터에 재사용한 것이다.

워크드 예제: “결과가 달라졌다”를 4요소로 분리하는 절차

표 9.1이 실제로 쓰이는 장면은 재현 성공이 아니라 **재현 실패의 원인 분리**다. “지난달과 같은 설정으로 돌렸는데 MAE가 다르다”는 신고가 들어왔다고 가정한다. 기록이 있으면 이것은 추리가 아니라 대조 절차가 된다 — 두 run을 나란히 놓고 요소별로 지운다.

파라미터 대조: 두 run의 기록된 파라미터를 필드별로 비교한다. 다르면 종결 — 애초에 같은 설정이 아니었다(이 경우가 생각보다 많다: “같은 설정”은 기억이고, 파라미터 기록은 사실이다).

데이터 지문 대조: 파라미터가 전부 같아도 `data_sha256_16` 이 다르면 데이터가 바뀐 것이다. 경로가 같아도 지문이 다르면 내용이 다르다 — 원천의 정정·추가(7장) 여부를 확인한다.

환경 대조: 파라미터·지문이 다 같다면 라이브러리 버전을 대조한다(run에 동봉된 의존성 기록). 수치 라이브러리의 마이너 버전 차이가 부동소수점 수준의 차이를 만드는 일은 실재한다.

난수 확인: 셋 다 같은데도 다르면, 훈련 코드에 seed가 고정되지 않은 난수가 있는지 확인한다 — 이 경우 “같은 설정”에 애초에 난수가 빠져 있던 것이고, 수정은 seed를 파라미터로 승격시키는 것이다.

이 절차의 요점은 순서다: 값싼 대조(기록 비교)를 먼저, 비싼 대조(환경 재구성·코드 검사)를 나중에. 그리고 절차 전체가 성립하려면 1~2단계의 기록이 훈련 시점에 이미 남아 있어야 한다 — 실패한 뒤에는 만들 수 없는 기록이다.

언제 불필요한가 — 반대 논거

탐색 초기, 폐기가 예정된 스케치 실험까지 전부 남기는 것은 기록 비용이 아니라 **소음 비용**이 문제다 — run 수백 개 속에서 의미 있는 것을 찾지 못하면 기록이 없는 것과 같다. 현실적인 기준은 다음과 같다: 기록 자체는 자동이므로 활성화해 두되(비활성화하면 유의미한 결과가 기록 밖에서 발생한다), 의미 있는 run에 태그·이름을 붙여 구분하고, 배포 후보만 레지스트리로 올린다(9.3). 즉 “모든 run을 기록”과 “선별된 후보만 등록”은 층이 다른 행위다 — 전자는 실험의 안전망, 후자는 운영의 관문이다.

기록의 **동기**는 도메인에 따라 다르게 말해진다. 공공에서 재현 4요소 기록은 감사·정보공개 청구에 답하는 이력이지만, 민간 ML 팀에서 같은 기록은 성능이 나빠졌을 때 원인을 빨리 찾고 팀원끼리 실험 설정을 주고받는 수단이다. 그래서 민간은 기록의 양 자체를 지표로 삼기도 한다 — 주당·월당 돌린 run 수를 실험 속도(experiment velocity)로 부르고, 이 수가 증가하는지를 팀 생산성의 대리 지표로 해석한다. 여기에도 앞의 층 구분이 그대로 적용된다: 기록을 자동으로 커 두어야 run을 빨리 돌릴 수 있고, 등록을 선별해야 그 run 더미가 배포 후보와 섞이지 않는다. 동기가 달라도 남겨야 하는 항목은 같고, 공공은 그 위에 “누가 승인했는가”가 한 줄 더 추가된다(9.4).

현장의 관점: 재현성 부재는 “기술 부채”다

이 문제를 산업 현장의 언어로 부르는 이름이 있다. Sculley 외(2015)의 “Hidden Technical Debt in Machine Learning Systems”는 실제 ML 시스템에서 순수 학습 코드가 전체의 작은 조각일 뿐이고 나머지 대부분이 데이터·설정·기록을 연결하는 주변 코드(글루 코드·설정)라고 지적하며, 어느 설정·어느 데이터로 이 모델이 나왔는지를 추적하지 못하는 상태를 재현·감사가 불가능해지는 **부채**로 규정했다. 이 책 1장이 같은 문헌을 CACE·글루 코드의 각도로 인용했다면, 여기서는 재현성·설정 관리의 각도에서 방법론을 차용한다(공공 부문 직접 선례가 아니라 프로덕션 ML 일반의 관점 차용이다). 이 관점은 이 장의 실습에서 직접 확인된다: 모델 훈련 자체는 두 줄(fit·predict)이고,

파라미터·지문·태그를 남기고 버전을 등록하는 기록 코드가 그보다 훨씬 길다(실습 9.1의 관찰). “작업의 대부분이 모델링”이라는 인상과 달리 운영 가능한 모델의 작업은 대부분 이 연결 코드에 있으며, 재현성은 그 연결 코드를 부채가 아니라 자산으로 전환하는 첫 항목이다.

공공 사례

정보공개 청구나 감사에서 “이 모델의 판단 근거를 소명하라”는 요구를 받았다고 가정한다. 모델 파일만 있는 기관은 답할 수 없다 — 모델 파일에는 “어떻게 만들어졌는가”가 들어 있지 않다. 실험 기록이 있는 기관은 그 모델의 run을 열어 훈련 데이터의 지문·기간, 파라미터, 성능 지표, 훈련 일시를 제시한다. 재현성은 연구자의 결벽이 아니라 **책임 소재의 전제 조건**이다: 재현할 수 없는 모델은 검증할 수 없고, 검증할 수 없는 판단을 행정에 쓸 수는 없다. ### 핵심 정리 - 기록 없는 모델은 재현·감사·비교가 전부 불가능하다 — 그리고 기록하지 않는 것이 기본값이다. - 모델 = $f(\text{코드, 데이터, 파라미터, 환경})$ — 훈련도 함수이며, 난수는 seed로 입력에 편입시킨다. - 데이터의 동일성은 도구가 자동 보장하지 않는다 — 지문(해시)을 직접 기록한다.

9.2 파라미터, 메트릭, 아티팩트 기록

학습 목표

run의 세 기록 항목(파라미터·메트릭·아티팩트)을 구분하고 무엇을 어디에 넣는지 안다.
후보 2개의 실측 기록과 재현성 검증(재실행 동일)을 확인한다.

직관: run은 훈련 1회의 기록 단위다

MLflow의 기록 단위는 **run**이다 — 훈련 한 번이 run 하나이고, 실험(experiment)은 run들을 묶는 폴더다. run 하나에 들어가는 항목은 세 종류로 구분된다.

파라미터(params): 훈련의 입력 설정 — 모델 종류, 하이퍼파라미터(hyperparameter), 그리고 실습에서는 데이터 지문(`data_sha256_16`)과 표본 수까지. “무엇을 입력했는가”다.

메트릭(metrics): 훈련의 결과 수치 — 실습은 훈련 MAE. “무엇이 산출되었는가”다.

파라미터와 달리 학습 과정 중 여러 번(스텝별) 기록할 수 있다 — 에포크를 도는 모델이라면 손실 곡선이 스텝별 메트릭으로 남고, “언제부터 과적합이 시작됐나” 같은 질문이 기록에서 답해진다. 실습의 두 후보는 반복 학습이 없어 최종값 한 번만 기록한다.

아티팩트(artifacts): 결과 파일 — 모델 자체, 훈련 데이터 사본, 그림. 수치로 환원되지 않는 실물이다.

구분이 중요한 이유는 질의 방식이 다르기 때문이다: “학습률 0.01로 돌린 run 전부”는 파라미터 검색이고, “MAE 최소 run”은 메트릭 정렬이고, “그 run의 훈련 데이터를 확인한다”는 아티팩트 다운로드다. 셋을 한 run에 함께 두는 것이 기록 단위의 완결성이다. 하나 더 — `run_id`는 실행마다 새로 발급되는 식별자다. 본문의 실측을 재실행하면 메트릭·계수는 같아도(아래 재현성) `run_id`는 다르다 — 그래서 이 장은 `run_id`가 아니라 값을 인용한다.

경계 사례 하나로 구분을 구체화한다: 실습은 훈련 데이터를 **아티팩트로도**(CSV 사본) **파라미터로도**(sha256 지문) 기록한다. 중복이 아니라 용도 분리다 — 지문은 가볍고 비교 가능해서 “같은 데이터인가”의 즉답에 사용되고(9.1의 분리 절차 2단계), 사본은 무겁지만 실물이라 “그 데이터를 다시 확인한다”에 사용된다. 수치 요약으로 충분한 것은 파라미터·메트릭으로, 실물이 필요한 것은 아티팩트로 — 이것이 배치 기준이다.

기록을 직접 탐색하는 창구도 있다. 실습의 백엔드는 SQLite 파일이므로, `mlflow ui --backend-store-uri sqlite:///data/output/mlflow.db` 를 실행하면 브라우저에서 experiment 아래 run 목록·파라미터·메트릭 표를 정렬·비교할 수 있다 — run이 수십 개로 늘었을 때 표 9.2를 수작업으로 작성하지 않게 해 주는 것이 이 바다(보충의 “비교 리포트 자동화”가 그 연장선이다). 이 장의 본문은 UI 화면이 아니라 증거 파일의 값을 인용한다 — 화면은 편의이고, 검증 가능한 근거는 파일이다.

이 세 항목의 구분은 MLflow만의 발명이 아니라 실험 추적 도구들이 공유하는 골격이다. MLflow 공식 문서가 정의하는 tracking의 기록 대상이 정확히 이 세 종류(+태그)이며, Weights & Biases 같은 다른 도구도 run 단위로 config(설정)·metrics(지표)·artifacts(파일)를 나누어 기록한다 — 이름이 조금씩 다를 뿐 “입력 설정 / 결과 수치 / 결과 파일”의 삼분은 공통이다. 그래서 이 장에서 익힌 구분은 도구를 변경해도 그대로 적용된다(도구 종속이 아니라 개념의 이식성이다). 실습이 MLflow를 선택한 이유는 로컬 SQLite만으로 추적·레지스트리·서빙을 한 프로세스 안에서 끝낼 수 있어 교육 부담이 가장 작기 때문이지, 이 삼분이 특정 도구에만 있어서가 아니다.

워크드 예제: 후보 2개, 수작업으로 검산되는 기록

실습의 훈련 데이터는 8장 피쳐 원천(9행 = 3지역 × 3일)에서 만든 **(전일 건수 → 당일 건수) 쌍 6개**다 — 지역마다 3일치에서 2쌍씩 나온다. 강남(11680)은 8→6, 6→9이고, 마포(11440)는 5→5, 5→6이며, 관악(11620)은 5→5, 5→5다. 모두 7장 확정 집계에서 수작업으로 확인된다(예: 강남 7/1=8, 7/2=6, 7/3=9). 이 6쌍의 정규화 CSV에 sha256을 걸어 지문 96f6b6b3ce9dcdd1... 을 얻고, 모든 run의 파라미터에 넣었다.

후보는 둘이다: **베이스라인**(전체 평균을 그대로 예측 — DummyRegressor)과 **선형회귀**(전일 건수의 1차 함수). 실측 기록은 다음과 같다(`ch9_experiment_report.json`).

표 9.2 후보별 기록(실제 실행, MLflow 3.14.0)

기록 항목	후보 1: baseline_mean	후보 2: linear
파라미터(모델)	DummyRegressor, strategy=mean	LinearRegression, fit_intercept=True
파라미터(데이터)	n_samples=6, data_sha256_16=96f6b6b3ce9dcdd1	동일(같은 데이터)
메트릭 train_mae	1.0000	1.0455
학습된 계수	(평균 6.0)	slope 0.4091 , intercept 3.6818
아티팩트	훈련 CSV, 모델	훈련 CSV, 모델

이 표는 모두 수작업으로 계산된다 — 데이터가 6쌍뿐인 이유가 바로 이것이다. 베이스라인: y 의 평균은 $(6+9+5+6+5+5)/6 = 36/6 = 6.0$ 이고, 잔차 절댓값은 $|6-6|, |9-6|, |5-6|, |6-6|, |5-6|, |5-6| = 0, 3, 1, 0, 1, 1$ — 합 6, 나누기 6, MAE **1.0**. 선형회귀: 최소제곱 공식으로 $\bar{x}=17/3, \bar{y}=6, S_{xy}=3, S_{xx}=22/3$ 이므로 $\text{slope} = 3 \div 22/3 = 9/22 = 0.4091$, $\text{intercept} = 6 - (9/22)(17/3) = 81/22 = 3.6818$ — 실측 계수와 정확히 일치한다. 이 직선의 MAE도 표본 하나로 확인한다: $x=5$ 인 샘플의 예측은 $3.6818 + 0.4091 \times 5 = 5.7273$ 이라 실측 y 가 5면 잔차 0.7273, 6이면 0.2727이다 — 여섯 잔차를 모두 더해 6으로 나누면 실측 메트릭 **1.0455**가 재현된다(전체 계산은 증거 파일과 대조 스크립트에 있다). 그리고 주목할 결과: **더 복잡한 모델이 기준선에 미달했다**($1.0455 > 1.0000$). 6쌍의 데이터에서 유의미한 기울기를 학습할 수 없으므로 당연한 결과이고, 이 당연함이 9.3의 승격 게이트 실측을 사실대로 보여 준다 — “새 후보가 항상 우월하다”는 전제가 아니라, 기준선을 넘지 못한 후보가 반려되는 흐름을 그대로 관찰한다.

한 가지 단서를 붙인다: 여기의 MAE는 **훈련 데이터 위의 훈련 오차**다. 6쌍을 훈련/평가로 나누는 것은 무의미해서 분리하지 않았다 — 실무라면 승격 판정(9.3)의 근거는 훈련 오차가 아니라 **보류 (holdout) 데이터의 평가 지표**여야 한다. 이 실습의 관찰 대상은 모델 품질이 아니라 기록·등록 메커니즘이다.

재현성 실측: 같은 설정을 다시 돌리면

9.1의 주장 — 네 요소가 같으면 같은 모델 — 을 실측했다. 선형회귀 후보를 같은 데이터(지문 동일)·같은 파라미터로 한 번 더 훈련한 결과: `train_mae 1.0455 = 1.0455 (mae_equal=true)`, 계수 `slope 0.4091·intercept 3.6818 = 동일 (coef_equal=true)`. 7장에서 “재실행 산출물의 해시가 같다”였던 것이 여기서는 “재훈련 모델의 메트릭·계수가 같다”가 되었다 — 같은 원리의 모델판이다. 거꾸로 읽으면 이것이 **회귀 검사**가 된다: 아무것도 바꾸지 않았는데 메트릭이 달라졌다면, 네 요소 중 무언가(대개 데이터)가 몰래 바뀐 것이고, 지문 파라미터가 어느 쪽인지 즉시 구분해 준다 — 지문이 같은데 결과가 다르면 코드·환경·난수를, 지문이 다르면 데이터를 의심한다.

핵심 코드

기록은 run 컨텍스트 안에서 세 종류를 각자의 API로 남기는 것이 전부다.

```
with mlflow.start_run(run_name="linear") as run:
    mlflow.log_params({"model_type": "LinearRegression", "data_sha256_16": fp[:16]})
    mlflow.log_metric("train_mae", mae)
    mlflow.log_artifact(str(csv_path), artifact_path="training_data")
    model_info = mlflow.sklearn.log_model(model, name="model") # 반환값을 보관하라(9.3)
```

전체 코드는 *practice/chapter9/code/9-1-mlflow-tracking.py* 참고

공공 사례

“작년 12월의 예측 모델과 올해 3월 모델 중 무엇이 나았는가”라는 질문이 국정감사 수준에서 나올 수 있다. 기록이 있으면 이것은 두 run의 메트릭 비교 + 두 훈련 데이터의 지문·기간 대조로 답이 되는 사무적 질문이다. 기록이 없으면 “당시 담당자의 기억”이 유일한 근거가 되고, 그 담당자가 이동했으면 근거가 없다. 공공 시스템의 인사 이동 주기를 고려하면, 실험 기록은 편의가 아니라 **조직 기억의 최소 장치**다.

주의점이 하나 있다 — 두 모델의 메트릭을 비교하려면 **평가 데이터가 같아야** 비교가 성립한다. 12월 모델은 12월 데이터로, 3월 모델은 3월 데이터로 평가한 수치를 나란히 놓는 것은 모델 비교가 아니라 데이터 비교다. 기록 시스템만으로 평가 데이터가 다른 모델의 비교 오류를 방지할 수는 없다. 평가 데이터의 지문이 run에 남아 있으면 “이 두 수치가 비교 가능한가”를 사후에 판별할 수 있다. ### 핵심 정리 - run 한 장 = 파라미터(입력 설정) + 메트릭(결과 수치) + 아티팩트(파일) — 질의 방식이 달라 구분이 실용적이다. - 데이터 지문을 파라미터로 남기면 “어느 데이터였나”가 증명 가능해지고, 재현 실패의 원인 분리가 빨라진다. - 같은 설정 재실행 → 같은 메트릭·계수(실측) — 7장 해시 동일의 모델판이자, 몰래 바뀐 입력을 잡는 회귀 검사다.

9.3 모델 레지스트리 단계와 승인 흐름

학습 목표

run(실험 기록)과 등록 모델(배포 후보)의 분리를 이해한다.
버전·별칭(champion)·승격 게이트의 실측 흐름을 따라간다.

직관: 실험 기록과 배포 승인은 다른 층이다

실험을 계속하면 run은 수십·수백 개가 쌓인다 — 그것이 정상이다(9.1). 그러나 서빙 시스템에 “run 목록에서 적합한 것을 선택해 사용하라”고 할 수는 없다. 배포는 실험과 다른 규율을 요구한다: 후보는 **이름 있는 모델의 번호 있는 버전**이어야 하고, 지금 운영 중인 것이 어느 버전인지 한 곳에서 답해져야 한다. 이 관문이 **모델 레지스트리**다 — MLOps 참조 아키텍처가 공통으로 포함하는 표준 구성요소이며(Kreuzberger 외 2023 — 배경), 실험 추적과는 층이 다른 별도의 관리 층으로 다룬다(로컬 실습에서는 같은 mlflow.db 파일에 함께 담기지만, run 기록과 등록 버전은 논리적으로 구분되는 층이다). run은 개별 실행의 기록이고, 레지스트리는 “민원 건수 예측기”라는 이름 아래 배포 후보를 버전으로 관리하는 별도 저장소다 — 같은 이름 아래 v1, v2, v3가 누적되고, 각 버전은 자기를 만든 run을 가리킨다(기록과의 연결은 끊기지 않는다).

전통적으로 이 관문은 **단계(stage)**로 표현되었다 — 개발(None) → 검증(Staging) → 운영(Production). 개념은 여전히 유효하지만, MLflow의 현행 방식은 stage API가 아니라 **별칭(alias)**이다: 버전에 champion 같은 가변 이름표를 붙이고, 소비자는 버전 번호가 아니라 별칭으로 모델을 로드한다(`models:/이름@champion`). 별칭을 v1에서 v2로 옮기는 것이 곧 운영 모델 교체이고, 소비자 코드는 한 줄도 바뀌지 않는다 — 8장에서 서빙이 피처를 “정의 이름”으로 조회했듯,

여기서는 모델을 “역할 이름”으로 조회한다. 이것은 이 실습만의 편법이 아니다 — MLflow 공식 문서 자체가 현행 권장 배포 워크플로로 별칭(예: @champion)과 태그를 제시하며, 버전을 실험·검증·운영 단계로 승격시키는 표준 수단으로 안내한다(직접 선례: 이 실습의 방식이 곧 도구의 표준 사용법이다).

단계 개념과 별칭 구현의 대응을 정리한다 — 오래된 자료·타 도구에서 stage 용어를 확인해도 번역할 수 있어야 한다.

표 9.3a 단계 개념과 별칭 구현의 대응

단계 개념(전통)	뜻	별칭 방식의 표현(현행)
개발(None)	등록만 된 후보	별칭 없는 버전(실습의 반려된 v2가 이 상태)
검증(Staging)	운영 투입 전 평가 중	challenger 같은 별칭을 관례로 부여
운영(Production)	서빙이 소비 중	champion 별칭 — 소비자가 이것만 바라본다
폐기(Archived)	더는 후보 아님	별칭 제거(버전 기록 자체는 남는다)

단계가 고정된 4칸이라면 별칭은 팀이 정의하는 이름표라는 것이 실질적 차이다 — 지역별 champion(champion-seoul)이든 실험 중 후보(challenger-2)든 운영 형태에 맞게 늘릴 수 있고, 대신 이름의 의미를 팀 규약으로 문서화할 책임이 생긴다. 이 장의 실습은 최소 규약(champion 하나)만 쓴다. 운영 규모가 커지면 이 이름표 규약이 곧 관리 체계가 된다 — 모델 수십·수백 개를 동시에 운영하는 민간 조직은 별칭을 배포 스위치(피쳐 플래그)와 묶어, 어느 모델의 어느 버전이 어느 트래픽에 걸려 있는지를 한 곳에서 조회한다. 공공은 모델 하나의 이력을 길게 관리하는 쪽이라 이름표 수는 적지만, “지금 운영 중인 것이 무엇인가”에 사람이 아니라 시스템이 답해야 한다는 요구는 같다.

워크드 예제: 등록·게이트·반려 실측

실습은 등록 모델 complaint_daily_forecaster 를 만들고 두 후보를 차례로 등록했다. 승격 규칙은 두 조건이다: ① 절대 게이트 — $\text{train_mae} \leq 1.05$, ② 상대 조건 — 기존 champion보다 개선. 실측 판정은 다음과 같다(ch9_promotion_record.json).

표 9.3 레지스트리 실측(등록·게이트·별칭)

버전	후보	train_mae	게이트(≤ 1.05)	champion 대비	판정(실측)
v1	baseline_mean	1.0000	통과	(최초 — 비교 대상 없음)	champion 별칭 부여 (승격)
v2	linear	1.0455	통과	1.0455 > 1.0000, 개선 없음	승격 반려 — v2로 등록은 유지

v2의 판정 결과는 다음과 같다 — 게이트는 통과했다($1.0455 \leq 1.05$). 탈락 사유는 상대 조건이다: 지금 운영 중인 것보다 나아야 교체할 이유가 있다. “절대 기준 통과”와 “교체 가치”는 다른 판단이며, 승격 규칙이 둘 다 명시해야 하는 이유다. 또 하나: 반려된 v2는 삭제되지 않고 **버전으로 남는다**. 이것은 낭비가 아니라 감사 기록이다 — “선형회귀를 시도했고, 이 수치로 반려되었다”가 레지스트리에 있으면, 다음 담당자가 같은 시도를 반복하기 전에 판정 이력을 확인할 수 있다.

이 “더 단순한 모델이 더 낫다”는 관찰은 이 실습이 우연히 만든 사례가 아니라 산업 현장이 명시적으로 권고하는 점검 항목이다. Breck 외(2017)의 ML Test Score는 프로덕션 ML의 준비도를 28개 테스트로 채점하는 루브릭인데, 그 모델 항목 중 하나가 “더 단순한 모델이 더 낫지 않은지 정기적으로 시험하라 — 예컨대 피처가 매우 적은 선형 모델 같은 아주 단순한 기준선과 비교해”이다(공공 부문 직접 선례가 아니라 프로덕션 ML 일반의 방법론 차용). 요지는 후보 모델이 단순한 기준선을 실제로 이기는지 늘 확인하라는 것이다. 이 실습은 그 점검을 수행한 사례가 되었다 — 더 단순한 평균 베이스라인(v1)과 그보다 복잡한 선형 후보(v2)를 비교하자 단순한 쪽의 오차가 더 작았다. 루브릭이 “혹시 단순 모델이 더 낫지 않은가”를 물으라 한 바로 그 상황이 실측으로 나타난 것이고, 승격 반려는 그 물음에 대한 정직한 답이다. 같은 루브릭은 “모델이 재현되는가”도 별도 테스트로 요구하는데, 이 장이 승격을 성능 게이트만으로 끝내지 않고 재현성 확인을 승격 검수 항목으로 함께 둔 것(표 9.4의 재현성 확인 행)이 그 요구의 실습판이다.

승격 후의 소비를 실측한다. `models:/complaint_daily_forecaster@champion` 으로 로드하면 v1(베이스라인)이 오고, 8장 온라인 스토어의 강남구 최신 피처(전일 건수 9 — 8장 표 8.2b의 실측값)를 입력하면 예측은 **6.00**이다. 계산: champion은 평균 예측기이므로 입력과 무관하게 훈련 y 평균 $36/6 = 6.0$ 을 반환한다 — 실측과 일치. 입력 9가 예측에 반영되지 않는 것은 버그가 아니라 v1의 정의다(그리고 이 무반응이야말로 v2 같은 개선 시도가 계속되어야 할 이유다).

게이트 값은 어디서 오는가

실습의 규칙 두 줄($\text{train_mae} \leq 1.05$, champion 대비 개선)은 주어진 값이 아니라 네 가지 선택의 결과다. 첫째, **어떤 지표로 판정하는가**. MAE를 고르면 큰 오차 하나와 작은 오차 여럿을 같은 무게로 세웠다고 선언한 것이다. 민원 급증 예측처럼 한 번의 큰 빗나감이 인력 배치 실패로 이어지는 업무라면 제곱 오차 계열(RMSE)이 그 실패를 더 무겁게 센다. 지표 선택은 통계 취향이 아니라 **업무 손실의 모양을 고르는 일**이다. 둘째, **임계값을 어디서 도출하는가**. 1.05라는 수치에 근거를 제시하려면 “지금 사람이 하는 방식의 오차가 얼마인가” 또는 “오차가 얼마를 넘으면 현장 배치가 틀어지는가” 중 하나를 먼저 말해야 한다. 그 둘 중 어디에도 닿지 않는 임계값은 반올림한 관행일 뿐이다.

셋째, **단일 조건인가 복합 조건인가**. 실습은 절대·상대 두 조건만 걸었지만 운영에서는 조건이 더 붙는 것이 보통이다 — 재현성 확인(9.4), 훈련 데이터의 지문이 승인된 원천과 일치하는지, 특정 지역구에서만 오차가 커지지 않았는지(집단별 오차 격차, 16장). 조건을 늘릴수록 승격은 어려워지고, 교체가 미뤄져 오래된 모델이 계속 운영에 남는다. 조건을 줄이면 교체는 빨라지지만 “우연히 좋은 run”이 운영에 들어간다. 넷째, **게이트가 막았을 때 누가 예외를 승인하는가**. 예외 경로가 없으면 급한 상황에서 게이트 임계 자체를 조용히 낮추게 되고, 예외 경로가 있으면 그 사용 이력이 감사 대상으로 남는다 — 관리 가능한 쪽은 후자다.

민간이 같은 자리에서 쓰는 값은 예측 정확도가 아니라 사업 지표다. 추천·광고 모델의 1차 판정 지표는 클릭률(CTR, click-through rate)이고, 승격 게이트는 대체로 “대조군 대비 CTR이 통계적으로 유의하게 올랐는가”가 된다. 다만 CTR 단독 게이트에는 알려진 한계가 있다 — 클릭만 늘리고 실제 구매·가입은 줄이는 모델이 통과한다. 그래서 전환율(CVR, conversion rate)을 함께 걸어 두 지표가 동시에 나빠지지 않을 것을 요구하는데, 이것이 이 장의 “절대 기준 + 상대 기준” 이중 게이트와 같은 구조다. 한 겹 더 들어가면 사용자당 평균 매출(ARPU, average revenue per user)까지 보고, 여기서 공공과 민간이 공유하는 문제가 드러난다 — **훈련 지표가 좋아지는 것과 목적이 달성되는 것은 다르다**. MAE가 내려가도 상담 인력 배치가 나아지지 않을 수 있고, CTR이 올라도 매출은 그대로일 수 있다. 게이트에 무엇을 걸 것인가는 결국 “이 모델이 왜 필요했는가”로 되돌아가는 질문이다.

별칭의 다른 얼굴: 롤백(rollback)

별칭 이동이 승격이라면, 같은 동작의 역방향이 **롤백(rollback)**이다. v2를 champion으로 올린 뒤 운영에서 문제가 드러났다고 가정한다 — 별칭을 v1로 되돌리는 한 번의 이동으로 서빙은 직전 모델로 복귀하고, 소비자 코드는 이번에도 바뀌지 않는다. 이것이 가능한 전제 두 가지가 이 장에서 이미 마련되었다: 반려·교체된 버전을 삭제하지 않고 보존하는 것(v1이 남아 있어야 복귀 대상이 있다), 그리고 각 버전이 자기 run(데이터 지문·파라미터)과 연결되어 있어 “무엇으로 돌아가는지”가 확실한 것. 7장에서 배치의 재실행 가능성이 복구의 전제였듯, 레지스트리에서는 버전 보존이 롤백의 전제다 — 배포 전략으로서의 롤백(트래픽 전환·헬스 체크)은 10장에서 이 위에 쌓인다.

실행 환경에서 발생한 오류: 등록 경로는 반환값에서 받는다

이 실습을 구성하며 등록 단계에서 실제로 발생한 오류를 기록해 둔다. 처음에는 버전 등록의 source를 관례적 경로 문자열 `runs:/<run_id>/model`로 조립해 넘겼다. 등록까지는 아무 에러가 없었다 — 그러나 별칭으로 로드하는 순간 `Could not find an 'MLmodel' configuration file` 예외가 발생했다. MLflow 3의 모델 저장 위치가 그 관례 경로와 달라졌기 때문인데, 요점은 특정 버전의 내부 구조가 아니라 실패의 형태다: **등록(쓰기)은 성공 로그를 남기고, 파손은 로드(읽기)에서야 드러났다**. 8장의 빈 온라인 스토어(적재는 성공, 조회에서 발현)와 같은 유형 — 오류가 난 곳과 원인이 있는 곳이 다르다. 수정은 경로를 수작업으로 조립하지 않고 `log_model`의 반환값(`model_info.model_uri`)을 등록에 전달하는 것이었고, 교훈은 두 겹이다: 도구가 반환하는 정보 참조를 사용한다(문자열 관례는 버전과 함께 구식이 된다), 그리고 등록 직후 **별칭 로드까지가 등록 검증**이다 — 실습 코드가 시나리오 4(로드·예측)를 반드시 통과해야 PASS인 이유다.

민간의 상시 실험 문화, 공공의 승격 결재

레지스트리·별칭·게이트를 왜 이렇게까지 갖추는지는 민간의 실험 문화를 보면 분명해진다. 추천·검색·랭킹처럼 모델이 매출에 직결되는 조직은 모델을 “한 번 만들고 끝”이 아니라 **상시 실험한다** — 수십 개 후보를 병렬로 훈련해 오프라인 지표(과거 데이터 위의 평가)로 1차로 거르고, 통과한 소수만 온라인 A/B 테스트(라이브 트래픽 분할)로 실제 지표를 재측정한 뒤, 우월한 것만 승격하는 파이프라인이 일상이다. 레지스트리와 승격 게이트는 이 파이프라인의 관문 장치다(1장 1.2.4에서 소개한 “두 모델을 라이브 트래픽으로 동시에 비교하는 실험 환경”이 바로 그 현장이다).

공공은 목적을 물려받되 수단을 바꾼다. 첫째, **A/B 대신 새도다** — 시민마다 다른 행정 모델을 태우는 것은 형평성·법적 문제 소지가 있으므로, 후보(challenger)는 전원에게 champion으로 응답하면서 예측을 로그로만 남겨 재평가된다. 반려된 v2가 삭제되지 않고 버전으로 남은 것(위)이 이 재심을 가능하게 하며, 10장이 이 실습의 v2를 바로 그 challenger로 재사용해 실측한다(같은 입력 전일 건수 9에 champion 6.0으로 응답하고, challenger 7.3636은 로그에만 남긴다). 둘째, **승격은 결재 문서가 된다** — 민간이 지표 개선을 승격의 충분조건으로 삼는 자리에서, 공공은 “누가 어떤 규칙으로 어떤 근거를 보고 통과시켰는가”를 감사 가능한 기록으로 남긴다(9.4의 승격 검수 기록). 즉 민간에서 실험 기록이 성능 최적화의 도구라면, 공공에서 같은 기록은 **결정의 정당성을 사후에 소명하는 결재 문서**다 — 도구는 같고, 기록에 요구되는 항목이 하나 더 많다.

공공 사례

“현재 운영 중인 심사 보조 모델은 정확히 무엇인가”라는 감사 질문을 가정한다. 레지스트리 없는 조직의 답은 서버에 배포된 파일의 타임스탬프 추적이 된다 — 그 파일이 어느 훈련에서 산출되었는지는 별도 조사가 필요하다. 레지스트리가 있는 조직의 답은 한 줄이다: champion 별칭이 가리키는 버전, 그 버전이 가리키는 run, 그 run의 데이터 지문·파라미터·메트릭. 교체 이력 역시 별칭 이동 기록으로 남는다. 발주·검수 문서라면 이렇게 물어야 한다 — 운영 모델의 버전을 시스템이 답하는가, 담당자가 답하는가. ### 핵심 정리 - 레지스트리는 run 더미에서 배포 후보를 분리하는 관문 — 이름·버전·별칭으로 “지금 운영 중인 것”이 항상 답해진다. - 승격은 절대 게이트 + 기존 champion 대비 개선의 이중 판정이며, 반려도 버전으로 남아 감사 기록이 된다(실측: v1 승격, v2 반려). - 등록의 검증은 로드까지다 — 쓰기 성공 로그에 의존하지 말고, 별칭 로드·예측까지 통과시킨다(실제 발생한 오류).

9.4 공공 모델 운영 메타데이터

학습 목표

공공 감사가 모델에 요구하는 메타데이터 항목과 그 기록 위치를 안다.
승격 검수 기록·등록 체크리스트를 실습 산출물로 확인한다.

핵심 개념: 감사는 성능표가 아니라 책임표를 요구한다

기술 조직의 기록(9.2~9.3)은 “어떻게 만들었고 얼마나 좋은가”에 답한다. 공공 감사는 거기에 더해 묻는다 — 누가 만들었고, 무슨 목적으로, 어느 기간의 어떤 출처 데이터로, 어떤 제한 아래 쓰이는가. 이 항목들은 성능과 무관하지만 책임과 직결되며, 나중에 소급 작성이 사실상 불가능하다 (담당자 이동, 기억 소실). 그래서 훈련 시점에 run의 태그(tag)로, 등록 시점에 모델 설명으로 함께 기록한다. 실습이 모든 run에 남긴 태그가 그 최소 집합이다: owner(민원데이터팀 — 가상 표기), purpose(지역구별 익일 민원 건수 예측·교육용), data_source(8장 피쳐 스냅샷 — 원천은 시뮬레이션 민원의 확정 집계), data_period(2026-07-01~2026-07-03). 이 태그들은 run에 붙어 mlflow.db에 기록되고 UI로 확인되며, 휘발성 식별자를 뺀 커밋용 JSON 리포트에는 run_id와 함께 신지 않았다 — 태그의 정본은 DB의 run이다. 8장 피쳐 정의서(표 8.4)와 같은 철학의 모델판이다 — 피쳐에 소유자·원천이 붙듯, 모델에도 붙는다.

승격 검수 기록: 판정을 문서로 남긴다

9.3의 승격 판정은 코드가 내렸지만, 공공 운영에서 승격은 기술 행위가 아니라 **승인 행위**다 — 누가, 어떤 규칙으로, 어떤 근거를 보고 통과시켰는가가 남아야 한다. 실습은 판정 직후 그 내용을 **모델 승격 검수 기록**(ch9_promotion_record.json)으로 자동 생성한다 — 정보 산출물이며, 수작업으로 쓰는 문서가 아니라 판정 로직의 실측 출력에서 추출된다(8장 피쳐 정의서를 registry에서 추출한 것과 같은 “문서를 실물에서 생성” 패턴).

표 9.4 모델 승격 검수 기록 양식(실습 자동 생성, 실측 값)

항목	실측 값	왜 필요한가
모델 이름 / champion	complaint_daily_forecaster / v1	“지금 운영 중인 것”의 정보 답
승격 규칙	train_mae $\leq$ 1.05 그리고 기존 champion 대비 개선	판정 기준의 사전 명시(사후 재량 차단)
판정 이력	v1 승격(1.0000), v2 반려(1.0455, 개선 없음)	반려 포함 전체 이력 — 감사 대상
재현성 확인	재실행 메트릭·계수 동일(true)	“우연히 좋은 run” 승격 방지
데이터 지문	96f6b6b3ce9dcdd1... (sha256)	훈련 데이터의 증명 가능한 동일성
검수자	검수 담당(가상 — 시뮬레이션 표기)	승인 행위의 주체
제한 사항	6쌍 극소 데이터 — 성능 수치는 메커니즘 관찰용	모델의 유효 범위 명시

재현성 확인 행에서는 승격 심사가 성능 지표뿐 아니라 **재현 가능성**도 요구함을 확인할 수 있다. 자동 메트릭 게이트는 train_mae $\leq$ 1.05와 champion 대비 개선의 두 조건이고(표 9.3), 재현성은 별도의 승인 검수 요건이다(등록 체크리스트 항목). 재현되지 않는 run은 게이트 수치가 좋아도 승격 대상이 아니다. 그 수치가 코드·데이터의 성질인지 우연의 산물인지 구분할 수 없기 때문이다.

검수 기록에서 가장 자주 비는 칸이 검수자다. 자동 게이트가 판정을 내리면 “시스템이 통과시켰다”로 종결하기 쉽지만, 행정 처분이나 자원 배분에 쓰이는 모델이라면 그 판정을 확인한 사람이 남아야 한다. 설계할 때 정할 것이 셋이다 — 게이트를 통과했을 때 승격을 자동으로 둘 것인가 사람 확인을 요구할 것인가, 게이트 미달인데 승격해야 하는 상황에서 누가 예외를 승인하는가, champion 별칭을 옮길 권한을 누가 갖는가. 셋째가 특히 실무적이다: 모델을 훈련하는 사람과 별칭을 옮기는 사람이 같으면 게이트는 자기 검사에 그친다.

모델 등록 체크리스트

등록·승격 전에 통과해야 할 항목을 이 장의 실측에서 도출해 정리한다 — 정보 산출물(모델 등록 체크리스트)이며, 실습 코드의 PASS 조건이 이 목록의 자동화판이다.

- ☐ run에 파라미터·메트릭·아티팩트가 완결되게 기록되어 있는가(9.2) — 특히 **데이터 지문**
- ☐ 운영 메타데이터 태그(소유·목적·출처·기간)가 있는가(9.4)
- ☐ 같은 설정 재실행으로 재현성이 확인되었는가(실측: 메트릭·계수 동일)
- ☐ 승격 규칙(절대 게이트 + 상대 조건)이 사전에 문서화되어 있는가(9.3)
- ☐ 등록 후 **별칭 로드·예측까지** 검증했는가(9.3의 실제 오류 — 쓰기 성공은 증거가 아니다)
- ☐ 승격 검수 기록이 생성·보존되는가(표 9.4)

언제 불필요한가 — 반대 논거

혼자 하는 탐색 단계의 프로토타입에까지 검수 기록·승인 워크플로를 요구하면, 절차가 실험 속도를 저해한다 — 9.1에서 “모든 run 기록”과 “선별 등록”을 구분한 것과 같은 층 구분이 여기에도 적용된다. 기준은 모델의 출력이 **자신 밖의 누군가에게 영향을 주는가**: 노트북 안에서 끝나는 실험이면 run 기록(자동·무비용)까지로 충분하고, 다른 사람·부서·시민이 그 예측을 소비하는 순간부터 레지스트리와 검수 기록이 필요해진다. 거꾸로 말하면 — 공공 서비스에 들어가는 모델에 “아직 프로토타입이라서”라는 이유로 검수 기록을 생략하는 것은 층을 혼동한 것이다. 배포되는 순간 그것은 프로토타입이 아니다.

모델 카드와의 관계(포인터)

표 9.4는 승격이라는 **결정**의 기록이다. 모델을 이용자·외부에 설명하는 문서로는 모델 카드(model card)가 별도로 있다 — 용도·한계·평가 조건·윤리적 고려를 담는 공개용 요약이며, 보충에서 포인터로 다룬다. 모델 카드는 Mitchell 외(2019)가, 데이터셋을 같은 방식으로 문서화하는 데이터시트(datasheet)는 Gebru 외(2021)가 제안한 정형 양식으로, 둘 다 공공 부문 전용은 아니지만 (방법론 차용) 용도·한계·평가 조건을 표준 항목으로 정형화한다는 점에서 이 장의 태그·검수 기록과 층이 맞는다. 관계는 다음과 같다: 실험 기록과 검수 기록이 있으면 모델 카드는 그 추출·요약이 되고, 없으면 모델 카드는 소급 창작이 된다.

공공 사례

민원 예측 모델을 다른 지자체가 이관받는 상황을 가정한다. 코드와 모델 파일만 넘기면 이관받은 쪽은 아무것도 검증할 수 없다 — 어느 데이터 기간으로 훈련되었는지, 자기 지역 데이터로 재훈련할 때 무엇을 기준으로 승격할지 알 수 없기 때문이다. 태그·검수 기록·체크리스트가 함께 넘어가면 이관은 “파일 전달”이 아니라 “운영 체계 전달”이 된다. 시스템 이관·유지보수 계약이 흔한 공공 환경에서, 운영 메타데이터는 문서화 의무가 아니라 **이관 가능성의 조건**이다.

역방향: 이 장의 기록 규율을 민간 현장에 옮기면

지금까지는 민간의 실험 문화를 공공 승격 절차로 옮겼다(9.3). 반대 방향도 성립한다 — 이 장에서 만든 기록·등록 규율을 그대로 민간 현장에 적용하면 무엇이 바뀌고 무엇이 유지되는지 세 현장에서 확인한다.

A/B 테스트 플랫폼과 레지스트리 연동에서는 별칭이 트래픽 라우팅의 스위치가 된다. 이 장에서 소비자 코드가 `models:/이름@champion` 한 줄로 모델을 로드했듯, A/B 플랫폼은 대조군에 champion을, 실험군에 challenger를 배정한다. 별칭 이동이 곧 승격이라는 구조는 그대로이고, 바뀌는 것은 이동의 방향이다 — 공공에서는 검수자의 승인이, 민간에서는 실험 결과의 통계적 유의성이 별칭을 옮긴다. 10장의 새도 배포는 이 구조에서 트래픽 분할만 뺀 형태다.

모델 비교 리포트 자동화는 표 9.2를 수작업으로 작성할 수 없게 되는 지점에서 시작한다. run이 수백 개가 되면 파라미터·메트릭을 자동 추출해 정기 리포트로 생성하고, 읽는 방향이 하나 더 생긴다 — 어떤 피처 조합·하이퍼파라미터가 사업 지표를 가장 올렸는가. 이때 파라미터를 구조화해 기록해 둔 효과가 나타난다. 자유 서술 로그에 적어 둔 설정은 나중에 집계되지 않는다.

데이터 지문은 민간에서 오히려 더 자주 쓰인다. 사용자 행동 데이터는 계절·프로모션·정책 변경으로 빠르게 변해서, “지난달 모델이 더 나았다”는 주장이 모델 비교인지 데이터 비교인지 지문이 없으면 구분할 수 없다 — 9.2에서 설명한 평가 데이터 불일치 문제와 같은 구조다.

공통 원칙 — 무엇을 게이트 지표로 삼을 것인가, 임계를 어디서 도출할 것인가, 어떤 항목을 태그로 남길 것인가, 별칭을 옮길 권한을 누구에게 줄 것인가. 이 네 결정은 어느 현장에서도 똑같이 내려야 하고, **근거만 도메인에 따라 달라진다**. 공공에서 그 근거가 감사·형평성·법적 의무라면, 민간에서는 매출·비용·실험 속도다. 방향은 여기서도 비대칭이다 — 공공용으로 갖춘 검수 기록과 지문 기록을 민간에 도입하면 제외할 것이 없다. 검수 기록은 사후 분석 자료가 되고, 지문은 실험 비교의 전제가 된다.

핵심 정리

공공 감사는 성능표 너머의 책임표(누가·왜·어느 데이터·어떤 제한)를 요구하며, 훈련 시점의 태그가 그 기록 위치다.

승격은 승인 행위 — 규칙·근거·판정·검수자를 담은 검수 기록을 판정 실물에서 자동 생성한다. 등록 체크리스트의 마지막 항목은 항상 “로드까지 검증”이다 — 이 장의 실제 오류가 남긴 규칙.

게이트 지표·임계·태그·별칭 권한이라는 네 결정은 도메인과 무관하게 내려야 하고, 근거만 도메인에 따라 달라진다.

실습 9.1 승격 게이트 설계와 MLflow 실험 기록·모델 등록 (★★★★☆☆)

이 실습은 **설계** → **실행** → **대조** → **위험 식별** 네 단계로 진행한다. 순서에 의도가 있다. MLflow를 어떻게 호출하는지에는 선택할 것이 없다 — 기록 API는 정해진 절차다. 반면 “무엇이 얼마나 좋아져야 운영 모델을 바꾸는가”는 전부 선택이고, 그 선택을 실행 결과를 본 뒤에 하면 규칙이 아니라 사후 합리화가 된다. 그래서 승격 규칙을 먼저 확정하고, 실행 로그의 실측 메트릭에 자기 규칙을 적용해 판정이 일치하는지 확인한다. 감사·검수는 그 뒤 4단계에서 다룬다 — 설계 근거와 실행 증거가 모두 확보되어야 검수 질문에 답할 수 있기 때문이다.

실습 목표

요구사항에서 승격 게이트 규칙(판정 지표·임계값·상대 조건·태그·승인 주체)을 스스로 정하고 근거를 적는다(설계).

8장 피처를 훈련 입력으로 후보 2개를 훈련하고 run으로 기록한다(파라미터·메트릭·아티팩트·데이터 지문·운영 태그).

재현성을 실측 검증하고, 레지스트리 등록 → 승격 게이트 → champion 별칭 → 별칭 로드·예측의 전 흐름을 실행한다(실행).

자기 게이트를 실측 메트릭에 적용해 스크립트의 판정과 대조하고, 어긋난 원인을 설명한다(대조).

입력 데이터

훈련 데이터는 8장 실습의 피처 원천 스냅샷(`complaint_daily_features.parquet`, 9행)에서 만든 (전일 건수 → 당일 건수) 6쌍이다 — 원천은 4~7장과 동일한 시뮬레이션 민원의 확정 집계이며, 소유자·검수자 표기도 가상이다. 데이터가 극소하므로 성능 수치는 모델 품질의 주장이 아니라 **기록·재현·등록 메커니즘의 관찰용**이다. 관찰 대상은 실제 MLflow 3.14.0 + scikit-learn 1.9.0 엔진의 동작이고, 인용 수치는 모두 실제 실행 산출물에서 산출된다.

1단계 설계 — 승격 게이트 설계표와 판정 예측

코드를 실행하기 전에 승격 규칙을 확정한다. 아래 요구사항을 읽고 표 9.5의 빈칸을 직접 채운다. 표 9.3의 규칙($\text{train_mae} \leq 1.05$, champion 대비 개선)은 완성된 예시 답안이므로 그대로 옮겨 적지 않는다 — 채운 뒤에 비교한다.

요구사항(가상 발주 조건) - 지역구별 익일 민원 건수를 예측해 다음 날 상담 인력 배치에 사용한다. 예측이 실제보다 낮으면 대기 시간이 늘고, 높으면 인력이 유휴 상태가 된다. - 재훈련은 월 1회다. 모델을 교체하면 상담 인력 운영 계획도 함께 바뀐다. - 감사 요구: 운영 모델이 바뀔 때마다 “왜

바꿨는가”에 답할 수 있어야 한다. - 훈련 데이터의 원천은 7장 확정 집계이며, 정정 재실행으로 과거 값이 바뀔 수 있다. - 담당자는 2년 주기로 바뀐다.

표 9.5 승격 게이트 설계표 — 학습자 작성용

항목	내가 정한 값	근거(요구사항의 어느 문장에서 나왔는가)
승격 판정 지표		
절대 게이트 임계값		
상대 조건(기존 champion 대비 개선 폭)		
재현성·데이터 지문 요건		
운영 태그 최소 집합		
아티팩트 보존 범위		
승인 주체와 예외 승인 경로		
이 게이트가 깨지는 조건		

작성할 때 지킬 것이 네 가지다. 첫째, 근거 칸에 “기본값이라서” 또는 “보통 그렇게 한다”를 쓰지 않는다 — 표 9.3의 1.05를 근거 없이 옮겨 적는 것도 같은 위반이다. 둘째, 임계값은 요구사항의 문장에서 도출한다: 지금 사람이 배치를 정하는 방식의 오차, 또는 오차가 얼마를 넘으면 인력 계획이 틀어지는지 중 하나에 닿아야 한다(9.3 “게이트 값은 어디서 오는가”). 셋째, 상대 조건에는 **얼마나** 좋아져야 하는지를 적는다 — “개선되면”과 “0.1 이상 개선되면”은 다른 규칙이고, 앞의 것은 소수점 셋째 자리의 변동으로도 운영 모델을 바꾼다. 넷째, “이 게이트가 깨지는 조건”은 양방향으로 검토한다 — 규칙이 느슨해 나쁜 후보를 통과시키는 쪽과, 엄격해 어떤 후보도 통과하지 못하는 쪽이 둘 다 실패다. 뒤쪽은 오래된 모델이 교체 없이 운영에 계속 남는 결과가 된다.

태그와 아티팩트 두 칸은 성격이 조금 다르다. 태그는 “2년 뒤 담당자가 이 모델을 물려받았을 때 무엇을 알아야 하는가”에서 역산하고(9.4), 아티팩트 보존 범위는 “감사가 다시 열어 봐야 하는 실물이 무엇인가”에서 역산한다(9.2의 지문 대 사본 구분). 둘 다 저장 용량이 아니라 사후 질문에서 나오는 값이다.

판정 예측: 실행 스크립트는 후보 2개(평균 베이스라인, 선형회귀)를 훈련해 차례로 등록한다. 실행하기 **전에**, 자기가 정한 게이트를 두 후보에 적용하면 어떻게 될지 적어 둔다.

	후보 1: baseline_mean	후보 2: linear
내 게이트로 예측한 판정		
그렇게 본 이유		
실제(3단계에서 기입)		

한 줄 더 적는다 — 이 실습이 끝났을 때 champion은 몇 번 버전일 것 같은가. 답이 “당연히 새 후보 (v2)”라면 무엇을 근거로 그렇게 보았는지도 함께 적는다.

2단계 실행 — 네 시나리오 실행

Python 3.10 이상(이 실습은 3.13에서 검증)이면 되고, 서버 상주가 없다 — 추적·레지스트리 백엔드는 로컬 SQLite(`sqlite:///data/output/mlflow.db`)다. 레지스트리는 파일 백엔드로는 동작하지 않고 DB 백엔드를 요구하므로, SQLite URI 지정이 이 실습의 환경 요점이다.

```
cd practice/chapter9
python3 -m venv venv && source venv/bin/activate
pip install -r code/requirements.txt
python run_chapter9.py
```

스크립트는 네 시나리오로 진행하며, 각각이 절 하나씩에 대응한다.

시나리오	내용	대응 절
1	후보 2개(베이스라인·선형) 훈련·기록 — 지문·태그 포함	9.2, 9.4
2	같은 설정 재실행 → 메트릭·계수 동일 확인	9.1, 9.2
3	v1·v2 등록, 게이트 판정, champion 별칭	9.3
4	별칭 로드 → 8장 온라인 피쳐 값으로 예측, 검수 기록 생성	9.3, 9.4

핵심 코드

등록과 승격에 필요한 규칙이다 — `source`는 반드시 `log_model`의 반환값에서 받는다(9.3의 실제 오류).

```
mv = client.create_model_version(name=MODEL_NAME, source=model_info.model_uri,
                                run_id=run_id) # 등록: 정본 경로 사용
client.set_registered_model_alias(MODEL_NAME, "champion", mv.version) # 승격 = 별칭 이동
loaded = mlflow.pyfunc.load_model(f"models:{MODEL_NAME}@champion") # 소비: 별칭 로드
```

전체 코드는 `practice/chapter9/code/9-1-mlflow-tracking.py` 참고

실행 중 확인 체크리스트

- ☐ 훈련 쌍이 6개이고 지문이 `96f6b6b3ce9dcdd1...` 로 출력되는가 — 데이터가 같으면 지문도 같다(재실행해도 동일)
- ☐ 시나리오 1에서 `baseline 1.0000 / linear 1.0455`가 나오는가 — 표 9.2의 손 계산과 대조
- ☐ 시나리오 2에서 “메트릭 동일=True, 계수 동일=True”인가

- 시나리오 3에서 v1 승격·v2 반려·champion=v1인가 — v2의 반려 사유가 게이트가 아니라 “개선 없음”임을 로그에서 확인
- 시나리오 4에서 입력 9.0 → 예측 6.00인가(= 훈련 y 평균, champion이 평균 예측기라는 계산)
- 마지막 줄이 CH9_RUN_PASS 인가
- 실행 전에 1단계 설계표와 판정 예측을 적어 두었는가(실행 후에 적으면 대조가 성립하지 않는다)

산출물 안내

산출물	내용	용도
mlflow.db	추적·레지스트리 백엔드(SQLite)	run·버전·별칭의 실체
ch9_experiment_report.json	훈련 쌍·run 기록·재현성·판정·예측	이 장 인용 수치의 증거 파일
ch9_promotion_record.json	모델 승격 검수 기록(정본 양식)	
training_pairs_*.csv	run 아티팩트로도 기록된 훈련 데이터	지문의 원문
mlruns/ (실습 폴더 바로 아래)	모델·아티팩트 실물 디렉터리	등록 버전이 가리키는 파일들
표 9.5 승격 게이트 설계표(학습자 작성)	판정 지표·임계·상대 조건·태그·승인 주체와 각각의 근거	승격 게이트 설계표 — 검수 자산, 3단계 대조 후 확정

mlflow.db와 mlruns/의 분업을 실물로 확인해 둔다 — DB에는 수치·파라미터·버전·별칭(조회 대상)이, mlruns/에는 모델 파일·아티팩트(다운로드 대상)가 있다. 8장의 registry(정의)와 store(값) 분리와 같은 구조이며, 9.3의 실제 오류는 정확히 이 둘 사이의 연결(등록된 버전이 가리키는 실물 경로)이 끊긴 사고였다. 두 산출물 모두 재실행하면 다시 생성되는 로컬 산출물이므로 저장소에는 커밋하지 않는다 — 커밋되는 증거는 JSON 리포트다.

2단계 실행 결과(실제)

실제 실행 콘솔 출력의 발췌다(run_id는 실행마다 다르므로 가려 두었다 — 독자의 실행에서는 다른 값이 나오는 것이 정상이고, 그 외 모든 수치는 같아야 한다).

```

[data] 훈련 쌍 6개, 지문 96f6b6b3ce9dcdd1...
== 시나리오 1: 후보 2개 실험 기록 ==
[run] baseline_mean: train_mae=1.0000 run_id=..... coef=None
[run] linear: train_mae=1.0455 run_id=..... coef={'slope': 0.4091, 'intercept': 3.6818}
== 시나리오 2: 같은 설정 재실행(재현성) ==
    메트릭 동일=True, 계수 동일=True
== 시나리오 3: 모델 등록과 승격 게이트 ==
    v1: champion 별칭 부여(승격) / v2: 승격 반려 - 절대 게이트 통과했으나 기존 champion 대비 개선
    없음 → champion=v1
== 시나리오 4: 별칭(champion)으로 로드·예측 ==
    입력(전일 건수)=9.0 → 예측=6.00 (champion=v1)
CH9_RUN_PASS

```

훈련 쌍 6개(강남 8→6·6→9, 마포 5→5·5→6, 관악 5→5·5→5), 지문 96f6b6b3ce9dcdd1... — 8장 피쳐 원천 9행에서 유도, 모두 7장 확정 집계와 일치.

run 기록: baseline train_mae **1.0000**, linear train_mae **1.0455**(slope 0.4091, intercept 3.6818) — 표 9.2.

재현성: linear 재실행 → mae 1.0455=1.0455, 계수 동일 — mae_equal=true, coef_equal=true.

레지스트리: v1 등록·게이트 통과·champion 부여, v2 등록·게이트 통과·**승격 반려**(champion 대비 개선 없음) → champion=v1 — 표 9.3.

별칭 서빙: `models:/complaint_daily_forecaster@champion` 로드, 입력(전일 건수) 9.0 → 예측 **6.00** = 훈련 y 평균(검산 일치).

실행 시간은 수 초 수준이며, 전 과정이 로컬 파일·SQLite 안에서 끝난다 — 기록·등록의 규율에 무거운 인프라가 필요하지 않다는 것이 8장에 이어 반복되는 관찰이다.

관찰

인용된 모든 수치가 수작업으로 재현된다: 평균 6.0과 MAE 1.0은 암산, 선형 계수는 최소제곱 공식(9/22, 81/22), 예측 6.00은 평균 예측기의 정의. 6쌍 데이터의 목적이 이것이다 — 엔진의 출력을 믿는 것이 아니라 검산한다.

“새 후보가 우월해 champion 교체”가 아니라 “새 후보가 반려되고 기록으로 남는” 흐름을 관찰했다 — 실무에서 더 흔한 쪽은 후자이며, 반려 이력이야말로 레지스트리의 감사 가치다. run_id는 실행마다 다르지만 메트릭·계수·지문은 같다(재실행 확인). 같은 이유로 증거 JSON에는 휘발성 run_id를 넣지 않았고, 그 결과 재실행한 증거 파일이 바이트 단위로 동일하다(실측 — 7장 “재실행 해시 동일”의 9장판). 무엇이 바뀌어야 정상이고 무엇이 바뀌면 사고인지의 경계가 곧 재현성의 경계다.

시나리오 4가 없었다면 등록 단계의 경로 파손(아래)을 끝까지 몰랐을 것이다 — 쓰기의 성공 로그는 읽기의 성공을 보증하지 않는다. 4장 유실, 6장 폐기, 7장 정제 drop, 8장 빈 적재·행 탈락에 이어, 이번 장의 침묵 실패는 “등록은 되었는데 로드가 안 되는 모델”이다.

이 장에서 훈련보다 기록·등록 코드가 길다는 것을 직접 확인한다 — 모델 훈련 자체는 두 줄(fit·predict)이고 나머지가 모두 규율이다. MLOps에서 모델링이 차지하는 비중을 축약해 보여주는 사례이며, 이 비율은 실무에서도 크게 다르지 않다.

실행 환경에서 발생한 오류 사례

9.3에 요약한 오류의 경과다. 버전 등록의 source를 관례 문자열 `runs:<run_id>/model` 로 넘겼을 때, 등록 API는 정상적으로 버전을 만들었고 아무 경고도 없었다. 실패는 시나리오 4의 별칭 로드에서 `MlflowException: Could not find an 'MLmodel' configuration file` 로 나타났다 — 등록된 버전이 가리키는 위치에 모델 실물이 없었던 것이다. 수정은

`mlflow.sklearn.log_model(...)` 의 반환값 `model_info.model_uri` 를 source로 쓰는 것 한 줄이었다. 일반 교훈: 도구의 내부 경로 규약을 문자열로 재현하지 말고 API가 반환하는 참조를 보관한다. 그리고 “등록 성공”의 정의를 로드 성공까지로 확장한다 — 이 실습의 PASS 게이트가 예측값 계산($6.00 = y$ 평균)까지 요구하는 이유다.

3단계 대조 — 내 게이트를 실측 메트릭에 적용하기

1단계 예측표의 “실제” 줄을 채운다. 실측 메트릭은 `baseline_mean 1.0000`, `linear 1.0455`이고(표 9.2), 스크립트 규칙에 따른 판정은 v1 승격·v2 반려다(표 9.3). 여기에 **자기가 정한 게이트를 다시 적용해** 판정이 같은지 확인한다.

절대 게이트만 걸었다면 임계를 1.05로 설정한 경우 두 후보가 모두 통과하고, 1.0으로 설정한 경우 linear만 탈락한다. 어느 쪽이든 절대 기준 하나로 “교체할 이유”를 판정하지 못한다는 것이 v2에서 드러난다 — 게이트를 통과했는데도 교체하지 않는 것이 정답인 경우가 있다.

상대 조건에 개선 폭을 적었다면 이번 실행에서는 판정이 달라지지 않는다. linear는 개선이 아니라 악화($1.0455 > 1.0000$)라 어떤 폭을 요구하든 반려다. 폭 규정의 효과는 반대 상황에서 드러난다 — 후보가 1.0000 대신 0.9990을 냈다면, “개선되면 승격”은 교체하고 “0.05 이상 개선되면 승격”은 교체하지 않는다. 인력 운영 계획을 바꿔 가며 얻을 만한 차이인가가 그 폭의 근거다.

예측이 “새 후보가 우월하다”였다면 무엇을 빠뜨렸는가. 복잡한 모델이 단순한 기준선보다 우월하다는 보장은 없다. 6쌍의 데이터에서 기울기를 학습하려 한 선형회귀는 평균 예측기보다 오차가 컸고, 이것은 예외가 아니라 ML Test Score가 정기 점검 항목으로 요구하는 상황 그대로다(9.3). 승격 게이트는 새 후보를 통과시키는 장치가 아니라, 교체할 이유가 없을 때 교체하지 않게 막는 장치다.

재현성·지문 요건을 게이트에 넣었다면 이번 실행에서는 둘 다 충족된다(`mae_equal=true`, `coef_equal=true`, 지문 `96f6b6b3ce9dcdd1...` 동일). 넣지 않았다면 지금 한 줄을 추가한다 — 이 요건은 통과할 때 아무 일도 하지 않고, 데이터가 몰래 바뀐 실행에서만 작동한다. 추가 비용은 재실행 한 번이고, 대신 로그에 아무 흔적도 남기지 않는 데이터 교체를 검출한다.

판정 지표를 MAE가 아닌 것으로 선택했다면 이번 실행의 증거로는 검증되지 않는다. 증거 파일에 기록된 메트릭이 `train_mae` 하나이기 때문이다. 이 경우 표 9.5의 “이 게이트가 깨지는 조건”에 “판정에 필요한 지표가 run에 기록되지 않으면 게이트가 작동하지 않는다”를 적는다 — 게이트 설계와 기록 설계가 붙어 있다는 것이 여기서 드러난다.

마지막으로 표 9.5의 “이 게이트가 깨지는 조건” 칸을 다시 쓴다. 처음에 막연히 적었다면 지금은 근거가 생겼다 — 예컨대 훈련 오차만 보는 게이트는 표본이 6쌍뿐인 이 실습에서 훈련 데이터에 과하게 맞춘 후보를 제외하지 못한다. **제출하는 설계표는 이 수정을 반영한 최종본이다.**

실습과 운영의 간극(축소 지점 목록)

축소 지점을 명시한다. 데이터 규모(6쌍 ↔ 수만 샘플), 평가(훈련 오차 ↔ 보류 데이터·교차 검증), 추적 서버(로컬 SQLite ↔ 팀 공용 서버·인증), 승격(스크립트 내 즉시 판정 ↔ 검수자 승인 워크플로), 서빙(같은 프로세스에서 로드 ↔ 10장의 API 서빙). 축소된 것은 규모와 주변 체계이지 규율이 아니다 — 기록 항목, 지문, 이중 게이트, 별칭 소비, 검수 기록은 어느 규모에서든 같은 형태다.

4단계 위험 식별 — 설계와 증거를 검수 언어로 옮기기

앞의 세 단계에서 설계 근거(1단계)와 실행 증거(2·3단계)를 모두 확보했다. 검수·감사가 요구하는 것은 그 둘을 이어 붙인 형태다 — 무엇이 잘못될 수 있는가, 그렇게 판단한 근거가 어느 산출물의 어느 값인가, 무엇으로 막는가. 앞 네 행은 실행 증거에서, 마지막 행은 1단계 설계에서 나온다.

위험	실습에서의 근거	대응 방향
재현 불가 모델	지문·파라미터 기록 + 재실행 동일 실측	4요소 기록을 훈련 코드에 상시 편입
우연한 좋은 수치의 승격	메트릭 게이트 통과 후 재현성 확인을 승인 검수 요건으로 추가 (표 9.4)	승격 규칙 사전 문서화 + 보류 데이터 평가
등록·실물 불일치	source 경로 오류 → 로드 실패 (실제 경험)	등록 후 별칭 로드·예측까지 검증
운영 모델 불명	champion 별칭이 단일 정답 제공	버전 고정 대신 별칭 소비, 이동 이력 보존
게이트 규칙 자체의 결함	1단계 표 9.5와 3단계 대조: 훈련 오차 단독 게이트는 6쌍 표본의 과적합을 거르지 못하고, 개선 폭을 안 정하면 0.001 차이로도 교체된다	판정 지표를 보류 데이터 평가로 교체, 상대 조건에 최소 개선 폭 명시, 별칭 이동 권한을 훈련 담당과 분리

셋째 열이 “모니터링한다” 같은 말로 채워지면 그 행은 검수 문서에서 유용하지 않다. 대응 방향에는 바꿀 규칙이나 바꿀 값이 들어가야 한다.

제출물 3종: ① 표 9.5 승격 게이트 설계표(3단계 수정을 반영한 최종본), ②

ch9_experiment_report.json, ③ 위 위험 식별 표. ①이 설계, ②가 증거, ③이 둘을 이은 검수 자산이다.

보충(전문가 트랙 포인터)

모델 카드와 데이터셋 카드: 표 9.4(내부 검수 기록)의 외부 설명판 — 용도·한계·평가 조건·윤리 고려를 이용자 관점으로 요약한다. 실험 기록이 있으면 추출이고, 없으면 소급 창작이다(9.4).

승인 워크플로와 감사 로그: 실습의 즉시 판정을 운영에서는 검수자 승인 단계(별칭 이동 권한 분리)와 이동 이력 로그로 확장한다 — “누가 언제 champion을 옮겼나”가 감사 질문이 된다.

여러 모델 후보의 비교 리포트 자동화: run이 수십 개가 되면 표 9.2를 수작업으로 작성할 수 없다 — 실험 단위로 파라미터·메트릭 표를 자동 추출해 정기 리포트로 생성하는 것이 다음 단계다(파라미터·메트릭을 구조화해 기록해 둔 효과가 여기서 나타난다).

핵심 용어

실험 추적: 훈련 1회분의 입력·결과·산출물을 자동 기록하는 체계

experiment / run: 실험 폴더 / 훈련 1회의 기록 단위 — run이 파라미터·메트릭·아티팩트·태그를 담는다

run_id: run의 고유 식별자 — 실행마다 새로 발급되므로 본문 수치의 인용 근거로 사용하지 않는다

파라미터 / 메트릭 / 아티팩트: 입력 설정 / 결과 수치 / 결과 파일 — 질의 방식이 다르다

데이터 지문: 훈련 데이터의 해시를 파라미터로 기록한 것 — “어느 데이터였나”의 증명(5·7장 지문의 재사용)

재현성: 코드·데이터·파라미터·환경(+seed)이 같으면 같은 모델 — 실측은 메트릭·계수 동일로 확인

모델 레지스트리: 배포 후보를 이름·버전으로 관리하는 관문 — run(실험)과 분리된 층 버전: 같은 이름 아래 누적되는 등록 단위 — 반려된 후보도 버전으로 남아 이력이 된다

별칭(alias): 버전에 붙는 가변 이름표(champion) — 구식 stage의 현행 대체, 소비자는 별칭으로 로드

challenger: 검증 중인 도전 후보에 붙이는 관례적 별칭 — 단계 개념의 Staging에 해당(표 9.3a)

롤백: 별칭을 이전 버전으로 되돌리는 것 — 버전 보존이 전제(승격과 같은 동작의 역방향)

승격 게이트: 절대 기준(메트릭 임계) + 상대 기준(기존 champion 대비 개선)의 사전 명시된 판정 규칙

승격 검수 기록: 판정의 규칙·근거·결과·검수자를 남긴 문서 — 판정 실물에서 자동 생성(정본 산출물)

운영 메타데이터: 소유·목적·데이터 출처·기간·제한 — 성능이 아니라 책임의 기록

검수 질문 한 줄 요약

발주·검수 문서에서 이 장은 세 질문으로 압축된다: ① 임의의 운영 모델에 대해 “어느 데이터·파라미터로 훈련했나”를 시스템이 답하는가(사람의 기억이면 실험 추적 부재), ② 운영 중인 모델의 버전과 교체 이력이 한 곳에서 답해지는가(레지스트리·별칭), ③ 승격의 규칙과 판정 기록이 사전

명시·보존되는가(검수 기록). 셋 다 없으면 그 시스템의 모델 교체는 관리되는 절차가 아니라 개인의 손작업이다.

다음 장 예고

champion 별칭까지 왔지만, 모델은 아직 실습 스크립트 안에서만 로드되었다. 실제 이용자는 스크립트를 실행하지 않는다 — 모델은 API 뒤에 놓여 요청을 받아야 하고, 그 API는 컨테이너로 포장되어 검증 파이프라인을 통과한 뒤 배포되어야 한다. 10장은 이 배포의 층을 다룬다: 배치/온라인 추론의 선택, 카나리-shadow-blue-green 배포 전략, FastAPI 서빙과 Docker, GitHub Actions 검증 흐름. 이 장의 레지스트리가 그대로 이어진다 — 서빙 API가 로드하는 것이 바로

`models:/...@champion`이며, 별칭 이동이 배포 전략의 한 축임을 확인한다.

참고문헌

- MLflow. (n.d.). *MLflow Tracking*. <https://mlflow.org/docs/latest/ml/tracking/>
- MLflow. (n.d.). *MLflow Model Registry*. <https://mlflow.org/docs/latest/ml/model-registry/>
- MLflow. (n.d.). *Getting Started with MLflow*. <https://mlflow.org/docs/latest/ml/getting-started/>
- MLflow. (n.d.). *Tracking API*. <https://mlflow.org/docs/latest/ml/tracking/tracking-api/>
- Kreuzberger, D., Kühl, N., & Hirschl, S. (2023). *Machine Learning Operations (MLOps): Overview, Definition, and Architecture*. IEEE Access. <https://arxiv.org/abs/2205.02302>
- Sculley, D., Holt, G., Golovin, D., et al. (2015). *Hidden Technical Debt in Machine Learning Systems*. NeurIPS 2015. https://papers.nips.cc/paper_files/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf
- Breck, E., Cai, S., Nielsen, E., Salib, M., & Sculley, D. (2017). *The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction*. IEEE Big Data 2017. <https://research.google/pubs/the-ml-test-score-a-rubric-for-ml-production-readiness-and-technical-debt-reduction/>
- Weights & Biases. (n.d.). *W&B Documentation*. <https://docs.wandb.ai/>
- Mitchell, M., Wu, S., Zaldivar, A., et al. (2019). *Model Cards for Model Reporting*. FAT* 2019. <https://arxiv.org/abs/1810.03993>
- Geburu, T., Morgenstern, J., Vecchione, B., et al. (2021). *Datasheets for Datasets*. Communications of the ACM. <https://arxiv.org/abs/1803.09010>